

VibraGuard — Project Brief

Semester 3 Engineering Project • Predictive Maintenance • Adaptive VMD + Machine Learning + Explainable AI

Purpose of this document: Give Trae a clear product/engineering brief so it can generate the project file structure, frontend UI, backend APIs, ML pipeline integration points, and development requirements.

1. Project Vision

VibraGuard is a real-world-inspired industrial machine health monitoring system. It accepts vibration data from rotating machinery, processes the signal using Variational Mode Decomposition (VMD), extracts condition indicators, predicts machine/fault condition using a trained ML model, and presents the result through an attractive engineering dashboard.

The application should feel like a professional predictive-maintenance product rather than a basic college ML demo. The UI should clearly show machine health, vibration trends, detected faults, confidence, signal/IMF visualizations, and explanations for why a prediction was made.

2. Core System Pipeline

Input → Preprocessing → Adaptive VMD → IMF Generation → Feature Extraction → ML Prediction → XAI → Dashboard

The first implementation should support uploaded vibration datasets/CSV files and simulated or prerecorded sensor streams. The architecture should leave a clean integration point for future Raspberry Pi/ESP32/industrial sensor input.

3. Main Technical Modules

Module	What it should do
Data Input	Upload CSV vibration data; validate columns, sampling rate and missing values.
Signal Processing	Filtering, normalization, windowing and basic signal statistics.
Adaptive VMD	Optimize VMD parameters K and alpha rather than relying only on manually fixed values.
Feature Extraction	Calculate RMS, variance, kurtosis, skewness, crest factor, peak-to-peak, spectral entropy, dominant frequency and
ML Model	Train/save a fault-classification model and expose prediction + probability/confidence.
XAI	Use SHAP or an equivalent explanation layer to identify influential features/IMFs.
Monitoring	Show health score, fault status, confidence, trends and recent events.
Future Edge Layer	Keep inference API/model format suitable for later Raspberry Pi/ESP32 deployment.

4. UI / UX Requirements

The frontend should have a modern industrial-tech visual identity: dark dashboard, clean cards, subtle glow/animation, engineering-style charts, responsive layout, and clear warning/severity states. Avoid making it look like a generic admin template.

Dashboard: machine selector, live/last signal status, overall health score, current condition, prediction confidence, RPM/load fields when available, recent alerts, and vibration trend.

Signal Analysis: raw vibration waveform, frequency spectrum/FFT, filtered signal, and interactive zoom/hover.

VMD Analysis: display IMF waveforms and corresponding frequency/energy information. Allow the user to compare original signal versus decomposed components.

Prediction: fault class, confidence/probability distribution, severity, recommended action, and timestamp.

Explainability: SHAP-style feature importance plus an IMF-level explanation showing which mode/features contributed most to the decision.

Training: dataset upload, preprocessing configuration, train/validation metrics, confusion matrix, classification report, model version and saved-model status.

5. Suggested Pages

- **Overview Dashboard** — machine health and alerts.
- **Signal Lab** — upload and inspect vibration signals.
- **VMD Explorer** — K/alpha, optimized values, IMF plots and decomposition metrics.
- **Fault Diagnosis** — prediction, confidence, severity and explanation.
- **Model Training** — dataset selection, training controls and evaluation.
- **History / Reports** — previous predictions, trends and downloadable results.
- **System / Dataset** — model status, data format, sensor configuration and API health.

6. Recommended Technology Direction

Frontend: React + Vite + TypeScript, Tailwind CSS, a charting library such as Recharts/Plotly, and Framer Motion for restrained animations.

Backend: Python + FastAPI. Keep signal processing, VMD, feature extraction and model inference as separate services/modules.

ML: Python, NumPy, SciPy, scikit-learn; PyTorch/TensorFlow only where justified. Persist trained models with a reproducible preprocessing pipeline.

Data: Start with a suitable public bearing/rotating-machine vibration dataset. The system must clearly document the selected dataset, classes, sampling rate and train/test split.

7. Real-World Readiness

The project should not pretend to be a certified industrial monitoring system. Instead, build it with a realistic architecture: validated input, preprocessing consistency, model versioning, confidence thresholds, fault severity levels, logging, error handling, and a sensor-ready API. Later, a Raspberry Pi/ESP32 can send vibration windows to the FastAPI inference endpoint.

8. MVP vs Advanced Scope

MVP — must work	Advanced — add if time permits
CSV vibration upload	Live sensor/Raspberry Pi stream
VMD decomposition	Automatic PSO/GA VMD optimization
Feature extraction	Adaptive windowing

Trained ML classifier	DANN cross-domain adaptation
Fault prediction	RUL/degradation estimation if suitable data exists
Signal + IMF charts	3D/advanced visualization
Basic SHAP explanation	IMF-level XAI dashboard
Training/evaluation page	Edge INT8 deployment

9. What Trae Should Generate

Ask Trae to produce the complete initial project scaffold with a clean monorepo structure, frontend screens/components, backend API routes, ML module interfaces, data schemas, sample/mock data, environment configuration, README, and clear TODO/integration points. The UI should be functional with mock data before the trained model is connected.

Important: do not hard-code fake ML results and present them as real predictions. Use clearly labelled demo/mock mode until the trained model is connected.

10. Definition of Success

A user should be able to open VibraGuard, upload a vibration CSV, inspect the signal, run/observe VMD decomposition, extract features, obtain a trained-model fault prediction, see confidence and explanations, and review the result in a polished dashboard. The same backend architecture should later accept sensor-generated vibration windows.